Teaching a Web Agent to Verify Before Retrying: LoRA Fine-Tuning for Reliable Forum Posting

YuSong (mc554561), YuhuanYe (mc651922), JinBo (mc638880)
University of Macau

Abstract

Web automation tools may return an uncertain result even when an action has already succeeded. If an agent immediately retries, it can create duplicate posts. This report studies whether LoRA fine-tuning can teach a tool-using agent to verify an uncertain publishing result before retrying. We calibrate a simulator from real records and generate ShareGPT trajectories with a scripted reference policy. The planned comparison uses a base model, a LoRA adapter trained on normal trajectories, and one trained on normal and recovery trajectories. A preliminary few-shot check succeeds in seven of ten uncertain-state scenarios, but none of three repeated-uncertainty scenarios. The main evaluation will measure verification-first behavior, duplicate-post rate, recovery success, and normal-task performance. Results that are not yet available remain explicitly pending.

1 Introduction

Web agents operate in environments where an action can be executed without producing a reliable confirmation. This problem is particularly visible in browser-based forum posting: a post may be submitted successfully, while the client receives a timeout, a missing response, or a page state that has not yet reflected the new post. If an agent interprets this outcome as a failure and submits again, the result may be a duplicate post.

This project studies whether a language-model agent can learn a safer recovery policy. The central question is not only whether the agent can call a posting tool, but whether it can reason about an uncertain outcome and use an observation tool before deciding to retry. We focus on a controlled Tieba posting task with three tools: candidate retrieval, recent-post lookup, and publishing.

The project addresses three research questions:

- **RQ1:** Does LoRA fine-tuning increase the rate at which the agent verifies an uncertain result before retrying?
- **RQ2:** Does the learned recovery behavior reduce duplicate posts while preserving normal posting performance?
- **RQ3:** Does a strategy learned in a simulator calibrated from real records remain usable in a real browser environment?

Our design separates real observations from training trajectories. Real records are used to estimate environment parameters such as visibility delay and status frequencies. These parameters configure a simulator with explicit environment truth, controllable failures, and reproducible resets. Training trajectories are generated in this simulator and converted into ShareGPT format for LoRA fine-tuning. Real trajectories are reserved for few-shot feasibility checks and limited final evaluation.

The main contributions are:

1. a simulator for uncertain web publishing outcomes with delayed observations and explicit environment truth;
2. a three-condition evaluation design that separates general tool-format learning from recovery-policy learning; and
3. an evaluation protocol centered on behavior-level metrics, including verification-first rate, duplicate-post rate, and recovery success.

2 Background and Related Work

Tool-using language agents. ReAct interleaves reasoning and environment actions so that new observations can inform later steps (Yao et al., 2023). Our project tests a specific decision within that cycle: whether an agent verifies an action whose result is uncertain before deciding to retry it.

Web and browser agents. WebArena provides realistic, reproducible websites for evaluating autonomous web agents (Zhou et al., 2024). Our task uses a real browser-backed posting tool as its motivating environment and isolates one failure mode: the mismatch between a successful submission and its delayed or missing confirmation. The simulator preserves the tool observations and hidden state needed to evaluate that recovery decision.

Trajectory supervision and tool-use learning. ReAct also motivates representing interaction as alternating actions and observations (Yao et al., 2023). We encode each trajectory as a ShareGPT conversation with system, human, assistant tool-call, tool-result, and final-answer messages. Recovery supervision focuses on the ordering and consequences of verification and retry actions.

Parameter-efficient fine-tuning. LoRA adapts a pretrained model by learning low-rank updates while keeping the base model fixed (Hu et al., 2022). QLoRA further reduces memory requirements through quantized training (Dettmers et al., 2023). We use LoRA because the project asks whether a targeted recovery behavior can be learned without full-model fine-tuning.

Positioning of this work. Existing work has studied tool use, browser interaction, trajectory supervision, and parameter-efficient adaptation separately. Our project combines these ideas around uncertainty-aware recovery: after a publishing call returns `unknown`, the agent must distinguish delayed visibility from non-publication using only the available observation tool. This framing also makes it possible to compare model behavior against an environment-truth validator rather than relying on the agent's final self-report.

3 Detailed Description

3.1 Project Workflow

Figure 1 summarizes the project workflow. The real environment is used to calibrate the simulator, not to provide ordinary supervised training examples.

The workflow has seven stages: (1) collect real records in a controlled forum, (2) derive frozen environment parameters, (3) instantiate the simulator, (4) generate independent training and test scenarios, (5) convert successful trajectories to ShareGPT format, (6) train and evaluate LoRA adapters, and (7) perform limited real-world confirmation after health checks.

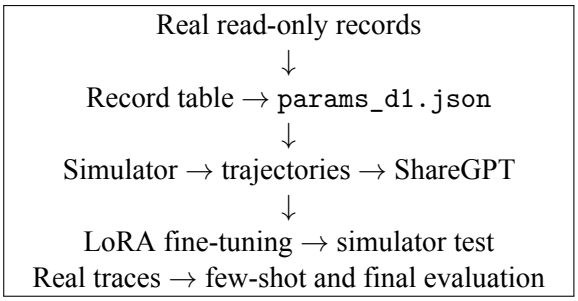


Figure 1: Data and evaluation workflow. Real records calibrate the simulator; simulated trajectories provide the main training data; real trajectories are reserved for few-shot checks and limited evaluation.

3.2 Task and Problem Formulation

The unified task is to select a candidate topic and publish it to a specified forum while checking for duplicates. The model has access to the following tools:

```
get_candidates()
list_recent_posts(kw, limit)
publish(kw, title, content)
```

The publishing tool can return `ok`, `unknown`, `rejected`, or `error`. The important case is `unknown`: the action may have succeeded, but the model does not receive a reliable post identifier or immediate confirmation. A subsequent list query may also fail to show the post because of visibility delay.

The reference policy uses a bounded verification budget and at most one republish attempt. After an uncertain result, the agent first queries recent posts. If the post is found, it stops. If it is not found and the verification budget remains, it queries again. Only after the available evidence supports non-publication may it republish once. The exact delay values are frozen from the D1 sampling procedure.

3.3 Real-World Data Collection and Boundary

Real sampling estimates four properties: the visibility delay after publication, the distribution of tool statuses, the title-length rule, and the behavior of duplicate submissions. The resulting parameter file is `params_d1.json`. Raw captures are stored separately with credentials redacted.

Data source	Use	Training?
Real records	Calibration	No
Parameters	Simulator setup	No
Sim. trajectories	Training/test	Yes
Real traces	Few-shot/evaluation	No
ShareGPT	LoRA input	Yes

Table 1: Data-use boundary in the project.

This boundary prevents real evaluation traces from becoming hidden positive demonstrations. It also makes the main quantitative comparison reproducible: the simulator exposes the same model-visible observation format while retaining an internal truth used only by the validator.

3.4 Simulator Design

The simulator, `MockTiebaEnv`, maintains an in-memory post database and supports explicit or probabilistic fault injection. Each trajectory starts from a clean environment. The simulator models the tool JSON format, the hidden post state, and delayed visibility through `list_recent_posts`. It does not attempt to reproduce the full Tieba interface, recommendation system, moderation process, or user comments.

Delayed visibility is essential. If a post became visible immediately after every uncertain response, the agent could learn the shortcut "query once; if absent, republish." The simulator therefore exposes observations that can remain ambiguous for a configurable number of queries. Training uses the frozen delay settings specified in the project plan, while test scenarios use separate candidate content and failure order.

3.5 Scenario Families and Splits

The generated simulated dataset contains 300 scenarios in each split, with 60 scenarios per family:

1. normal posting without a tool fault;
2. uncertain response with immediate visibility;
3. uncertain response with delayed visibility;
4. uncertain response where no post was created; and
5. uncertain response followed by an uncertain republish.

Candidate content is split into 60 training candidates, 25 test candidates, and 15 real-evaluation candidates, with no overlap between splits. Eighty of the 100 candidates were written for this project and are not real sampled posts. Each training candidate appears once in each of the five scenario families. Test scenarios use separate candidate content and failure settings.

3.6 Experimental Conditions

We compare three conditions:

Condition	Supervision	Purpose
A: Base	None	Baseline
B: Normal LoRA	300 normal	Format control
C: Recovery LoRA	60 + 240 recovery	Main condition

Table 2: Experimental conditions.

Condition B is necessary because normal trajectories may teach the model tool syntax and task formatting without teaching it how to recover from uncertainty. A stronger result for C than for B provides evidence that the model learned a recovery policy rather than merely becoming more familiar with the tools.

3.7 Trajectory Construction and LoRA Training

Each trajectory is converted to a ShareGPT conversation containing a fixed system prompt, the human task, alternating assistant tool calls and tool results, and a final answer. The system prompt is identical across A, B, and C and describes the task and output requirements without stating the recovery rule. The frozen B and C datasets each contain 300 scripted `ReferencePolicy` rollouts; C contains 60 normal and 240 recovery trajectories. Their labels were checked automatically against environment truth. Three separately reviewed, manually corrected demonstrations were used for the few-shot check and were not merged into C. The dataset provenance records the scripted origin; it should not be presented as model-generated or individually human-corrected data.

The frozen training configuration is Qwen3.5-9B with LoRA rank 8, alpha 16, all linear layers targeted, learning rate 5×10^{-5}, cosine scheduling with 0.1 warmup, 100 steps, batch size 1 with gradient accumulation 8, bf16 computation, gradient checkpointing, and disabled FlashAttention. Inference uses greedy decoding, a context length of 8192, at most 512 generated tokens, one tool call per turn, and a maximum of 16 tool steps.

3.8 Implementation Components

The implementation is divided into components with separate responsibilities:

- `MockTiebaEnv`: simulator state, delays, and fault injection;
- `TiebaAdapter`: common interface for simulated and real backends;
- Agent Runner: tool-call parsing, step limits, and trajectory logging;
- `convert_to_sharegpt.py`: conversion to training conversations;
- Validator: environment-truth trajectory classification; and
- `metrics_table.py`: aggregation of run manifests and metrics.

3.9 Testing and Validation

The validation plan has several layers. Simulator unit tests cover uncertain-state truth combinations, delayed visibility, reset behavior, and duplicate rules. Agent Runner tests check tool-call parsing and step limits. The validator is checked against reviewed traces, and an end-to-end run checks reproducibility under a fixed seed. The most recent progress record reports 104 passing tests and one pre-existing Windows-path test that fails on POSIX. A byte-for-byte comparison of simulated and real model-visible observations remains outstanding; therefore, interface equivalence should not yet be claimed.

4 Experimental Setup and Evaluation

4.1 Few-Shot Feasibility Gate

The base model was shown three reviewed recovery demonstrations and evaluated ten times on uncertain-state scenarios. The recorded result is seven successes out of ten, within the plan's 30%–80% continuation range. Success was 2/2 for delayed visibility, 3/3 for non-publication, and 2/2 for immediate visibility; it was 0/3 when the republish also returned an uncertain result. This small diagnostic is a feasibility gate, not the final model comparison.

The planned A/B/C evaluation uses the same 100 test scenarios for each condition, with 20 from each family. The Agent Runner, tools, system prompt, greedy decoding, and 16-step limit are held fixed.

4.2 Metrics

The primary behavioral metric is the **verification-first rate**: the proportion of uncertain trajectories whose first post-uncertainty action is `list_recent_posts` rather than `publish`. The main outcome metric is the **duplicate-post rate**, defined as the proportion of trajectories whose environment truth contains more than one post with the same content. We also report recovery success, normal-task success, average tool calls, average queries, and step-limit violations.

Trajectories are classified using environment truth rather than the model's final statement:

Class	Definition
`SUCCESS_NORMAL`	No fault and correct completion
`SUCCESS_RECOVERED`	Uncertainty handled with correct verification and final state
`FAILED_MODEL`	Incorrect decision, premature retry, or step-limit failure
`DUPLICATE_POST`	Environment truth contains a duplicate post
`FAILED_TOOL`	Login, selector, or tool-side failure

Table 3: Trajectory classification rules.

`FAILED_TOOL` trajectories are reported separately and excluded from the model-performance denominator because they do not measure the agent's decision policy.

5 Results

The A/B/C quantitative evaluation has not been reported in the available progress record. Table 4 reserves the final results without assigning values to unfinished runs.

Results should also be broken down by scenario family, especially delayed visibility, non-publication, and repeated uncertainty. C's benefit can be supported only if it improves verification-first behavior and duplicate-post rate relative to both A and B while preserving normal-task success.

6 Analysis and Discussion

6.1 Did the Agent Learn Verification?

We will compare the first action after each `unknown` response. A successful recovery policy should query before retrying, continue querying when the observation may be delayed, and republish only when the evidence supports non-publication. The analysis will include representative traces such as:

```
publish → unknown → list → list
→ found → stop
```

Condition	Verify-first	Duplicate rate	Recovery success	Normal success	Avg. calls
A: Base	–	–	–	–	–
B: Normal-only LoRA	–	–	–	–	–
C: Recovery LoRA	–	–	–	–	–

Table 4: Main simulator results. Dashes indicate pending evaluation results.

```
publish → unknown → list
→ absent → publish
```

These traces should be interpreted together with aggregate verification-first and duplicate-post rates.

6.2 Strategy Learning versus Tool-Format Learning

The B–C comparison is the key ablation. If B produces syntactically valid tool calls but still retries immediately, while C changes its behavior conditional on the observed state, the evidence supports recovery-policy learning rather than simple format adaptation.

6.3 Effect of Delayed Visibility

The analysis will examine whether the model behaves differently for immediate visibility and delayed visibility. The project deliberately distinguishes a post that is already created but not yet visible from a post that was never created. Cases with more than three unseen polls cannot always be distinguished from a swallowed post; this limitation will be reported rather than hidden.

6.4 Error Analysis

Failure cases will be grouped into premature republishing, incorrect stopping, excessive querying, and tool-side failures. For each group, we will report a compact trace and discuss whether the failure arises from the learned policy, the prompt/tool interface, or the simulator's abstraction.

7 Limitations, Difficulties, and Future Work

The planned real confirmation set contains only 5–10 runs and can support a limited migration check, not a statistically powered comparison. The simulator models delayed visibility and tool responses but not the full browser interface, moderation system, recommendation system, or changing page layouts. The experiment uses one base model, one account, one principal forum setting, and one frozen LoRA configuration. Moreover, a visibility delay larger than the polling window may be observationally indistinguishable from a swallowed post. The simulated/real observation-equivalence check is still incomplete.

The project also encountered practical difficulties: real browser state is affected by login status, page selectors, caching, rate limits, and account constraints; real traces cannot be freely reused for training; and tool failures must be separated from model failures. The C training set contains only successful scripted trajectories and no duplicate-post failures; the duplicate-post evaluation is therefore especially important. Future work could test broader fault families, multiple accounts and forums, longer delays, multiple random seeds, semantic duplicate detection, CAPTCHA-aware decisions, and calibrated stopping policies.

8 Conclusion

This report presents a controlled study of uncertainty-aware recovery for a tool-using web agent. The project combines real-record calibration, a delayed-observation simulator, ShareGPT trajectory supervision, and LoRA fine-tuning. The central experimental comparison is between a base model, a model trained only on normal trajectories, and a model trained on both normal and recovery trajectories.

The final conclusion will be based on whether recovery-specific supervision increases verification-first behavior, lowers duplicate-post rate, and preserves normal posting ability. The simulator provides the main quantitative evidence, while the real browser runs provide a limited check that the learned behavior does not immediately fail under real conditions. Because the real evaluation is intentionally small and the simulator is an abstraction, any claim about broad web-agent generalization should remain appropriately limited.

Acknowledgments

We thank the course instructors and project teammates for their feedback on the experimental design and evaluation protocol.

References

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*.